

|               |                                                |
|---------------|------------------------------------------------|
| Field         | Details                                        |
| Date          | 28-09-2026                                     |
| Team ID       | SWTID-2026-8733                                |
| Project Name  | Implement Client Script & UI Policy (Incident) |
| Maximum Marks | 4 Mark                                         |

## Phase-1 Brainstorming and Ideation

### Brainstorming

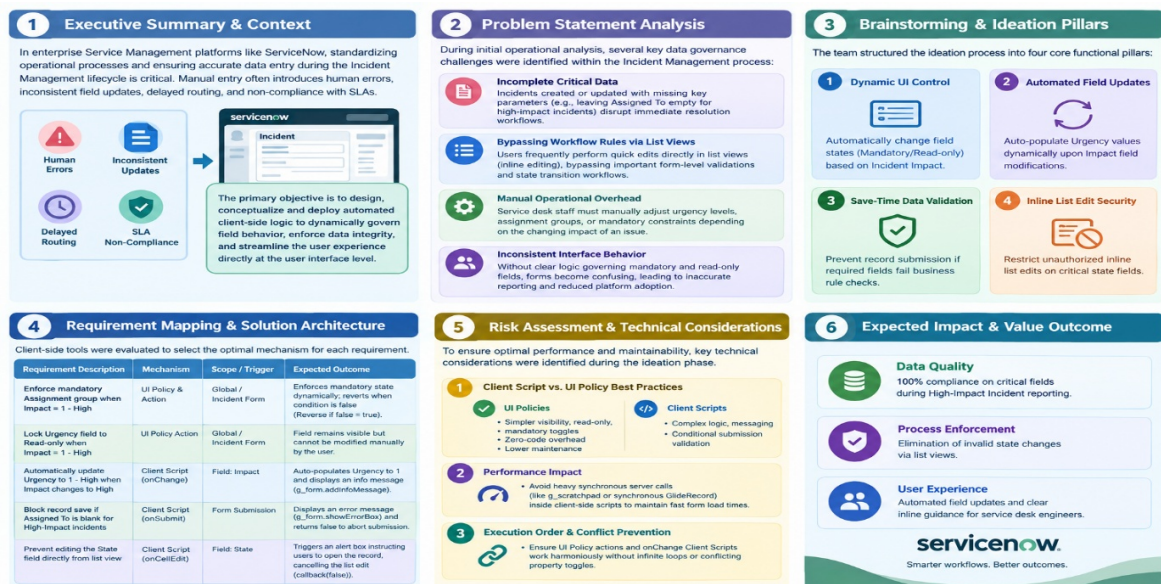
Brainstorming is the first step of the project development process. It is a creative process in which different ideas are discussed to identify a problem and find possible solutions. In this project, brainstorming is used to understand how form field behaviors, dynamic validations, and access controls can be managed on the Incident form using Client Scripts and UI Policies.

The main problem identified is that manual data entry on incident forms often leads to human error, incomplete mandatory details, and unauthorized inline editing directly from list views. For example, high-impact incidents are sometimes created or updated without assigning an Assignment Group or Assigned To user, leading to delayed resolution workflows.

During brainstorming, the following ideas were considered:

- Dynamically make the Assignment Group mandatory based on the Impact field value.
- Automatically update the Urgency field whenever the Impact field is modified.
- Validate key form fields upon submission to prevent saving incomplete incident records.
- Restrict direct inline list edits on critical fields like State to enforce proper form-level workflows.
- Combine UI Policies and Client Scripts to deliver a dynamic and responsive user interface without creating redundant server-side rules.

From these discussions, the combination of Client Scripts and UI Policies was identified as a suitable approach for providing flexible, real-time client-side controls and data validation on the Incident table.



## Phase 1: Brainstorming & Ideation Phase

### 1. Executive Summary & Context

In enterprise Service Management platforms like ServiceNow, standardizing operational processes and ensuring accurate data entry during the Incident Management lifecycle is critical. Manual entry often introduces human errors, inconsistent field updates, delayed routing, and non-compliance with Service Level Agreements (SLAs).

The primary objective of this project—**Implement Client Script & UI Policy (Incident)**—is to design, conceptualize, and deploy automated client-side logic to dynamically govern field behavior, enforce data integrity, and streamline the user experience directly at the user interface level.

### 2. Problem Statement Analysis

During initial operational analysis, several key data governance challenges were identified within the Incident Management process:

- **Incomplete Critical Data:** Incidents created or updated with missing key parameters (e.g., leaving Assigned To empty for high-impact incidents) disrupt immediate resolution workflows.
- **Bypassing Workflow Rules via List Views:** Users frequently perform quick edits directly in list views (inline editing), bypassing important form-level validations and state transition workflows.
- **Manual Operational Overhead:** Service desk staff must manually adjust urgency levels, assignment groups, or mandatory constraints depending on the changing impact of an issue.
- **Inconsistent Interface Behavior:** Without clear logic governing mandatory and read-only fields, forms become confusing, leading to inaccurate reporting and reduced platform adoption.

### 3. Brainstorming Core Requirements & Objectives

To solve these challenges, the team structured the ideation process into four core functional pillars:

|                                  |                                               |
|----------------------------------|-----------------------------------------------|
| +-----+                          |                                               |
|                                  |                                               |
| BRAINSTORMING & IDEATION PILLARS |                                               |
| +-----+                          |                                               |
| 1. Dynamic UI Control            | Automatically change field states (Mandatory/ |
|                                  | Read-only) based on Incident Impact.          |
| +-----+                          |                                               |
| 2. Automated Field Updates       | Auto-populate Urgency values dynamically upon |
|                                  | Impact field modifications.                   |
| +-----+                          |                                               |
| 3. Save-Time Data Validation     | Prevent record submission if required fields  |
|                                  | fail business rule checks.                    |
| +-----+                          |                                               |
| 4. Inline List Edit Security     | Restrict unauthorized inline list edits on    |
|                                  | critical state fields.                        |
| +-----+                          |                                               |

### 4. Requirement Mapping & Solution Architecture

During the ideation session, client-side tools were evaluated to select the optimal mechanism for each requirement:

| Requirement Description                                                     | Selected Mechanism                       | Scope / Trigger               | Expected Outcome                                                                                                 |  |
|-----------------------------------------------------------------------------|------------------------------------------|-------------------------------|------------------------------------------------------------------------------------------------------------------|--|
| Enforce mandatory Assignment group when Impact = 1 - High<br>PDF            | <b>UI Policy &amp; Action</b><br>PDF     | Global / Incident Form<br>PDF | Enforces mandatory state dynamically; reverts when condition is false (Reverse if false = true).<br>PDF          |  |
| Lock Urgency field to Read-only when Impact = 1 - High<br>PDF               | <b>UI Policy Action</b><br>PDF           | Global / Incident Form<br>PDF | Field remains visible but cannot be modified manually by the user.<br>PDF                                        |  |
| Automatically update Urgency to 1 - High when Impact changes to High<br>PDF | <b>Client Script (onChange)</b><br>PDF   | Field: Impact<br>PDF          | Auto-populates Urgency to 1 and displays an informational message (g_form.addInfoMessage).<br>PDF                |  |
| Block record save if Assigned To is blank for High-Impact incidents<br>PDF  | <b>Client Script (onSubmit)</b><br>PDF   | Form Submission<br>PDF        | Displays an error message (g_form.showErrorBox) and returns false to abort submission.<br>PDF                    |  |
| Prevent editing the State field directly from list view<br>PDF              | <b>Client Script (onCellEdit)</b><br>PDF | Field: State<br>PDF           | Triggers an alert box instructing users to open the record, cancelling the list edit via callback(false).<br>PDF |  |

## 5. Risk Assessment & Technical Considerations

During the ideation phase, key technical considerations were identified to ensure optimal performance and maintainability:

### 1. Client Script vs. UI Policy Best Practices:

- *UI Policies* are preferred for simple visibility, read-only, and mandatory toggles due to zero-code overhead and lower maintenance.
- *Client Scripts* are reserved for complex logic, messaging, and conditional submission validation.

### 2. Performance Impact:

- Avoiding heavy synchronous server calls (like g\_scratchpad or synchronous GlideRecord) inside client-side scripts to maintain fast form load times.

### 3. Execution Order & Conflict Prevention:

- Ensuring UI Policy actions and onChange Client Scripts work harmoniously without infinite loops or conflicting property toggles.

#### **6. Expected Impact & Value Outcome**

- **Data Quality:** 100% compliance on critical fields during High-Impact Incident reporting.
- **Process Enforcement:** Elimination of invalid state changes via list views.
- **User Experience:** Automated field updates and clear inline guidance for service desk engineers.